Formal Verification Report: Morpho Vault V2 (Valid State)

- Date: July 5th, 2026
- Audit Repo: https://github.com/alexzoid-eth/morpho-vault-v2-fv
- Client Repo: https://github.com/morpho-org/vault-v2
- Audit Commit: 549b113106e505b83952c7b296622534338d5f92 (June 23rd, 2026)
- Mitigation Commit: —
- Author: AlexZoid
- Certora Prover version: 8.16.2

Table of Contents

About Vault V2

Vault V2 (`src/VaultV2.sol`, solc 0.8.28) is an ERC-4626 + ERC-2612 vault that routes idle `asset` into pluggable **adapters**, gated by four external permission gates (`receiveSharesGate` / `sendSharesGate` / `receiveAssetsGate` / `sendAssetsGate`) and governed by owner / curator / allocator / sentinel roles with per-selector timelocks. Interest is accrued against a curator-set `maxRate`; performance and management fees are minted as shares to their recipients. Shares are minted through an inflation-resistant `virtualShares = 10**decimalOffset`. All contracts are immutable and the vault uses transient storage for the per-transaction `firstTotalAssets` marker.

The formal verification scope covers a single target:

1. **VaultV2** (`src/VaultV2.sol`) — the sole core contract; holds every role, cap, fee, and ERC-20 share balance. Its full external surface (ERC-4626 `deposit` / `mint` / `withdraw` / `redeem` / `forceDeallocate`, ERC-20 / ERC-2612 `transfer` / `approve` / `permit`, curator / allocator / owner config setters, `submit` / `revoke` timelock plumbing, `allocate` / `deallocate`, `accrueInterest`) is verified parametrically. Adapters, the adapter registry,

the four gates, and the underlying ERC-20 `asset` are out of scope — modeled as CVL summaries and reached only through VaultV2's own interface.

This run covers **valid-state properties only**; the other property categories are out of scope.

Formal Verification Methodology

Certora Formal Verification (FV) provides mathematical proofs of smart-contract correctness by verifying code against a formal specification. Unlike testing and fuzzing, which examine specific execution paths, Certora FV examines all reachable states and execution paths. Properties are written in CVL (Certora Verification Language) and checked by the prover against the compiled contract.

Verification Approach

VaultV2 is verified standalone — one Certora target, one harness (`VaultV2Harness`, which forwards the constructor), and one storage **ghost-mirror** model. Every VaultV2 storage variable is mirrored by a `persistent ghost ghostTv<Var>` kept in lock-step with storage via a paired `hook Sload` (`require ghost == value`) / `hook Sstore` (`ghost = value`). The share ledger (`totalSupply` / `balanceOf` / `allowance`) is the one exception: it is mirrored into the **shared ERC20 ghosts** (`ghostERC20TotalSupply256` / `ghostERC20Balances128` / `ghostERC20Allowances256`) keyed by the vault address `_VaultV2`, so the vault's own shares reuse the same bounded token model as the underlying `asset`. Invariants are expressed over these mirrors, so a downstream project can reuse them without re-deriving the storage layout (see Reuse / Import).

External dependencies are modeled in CVL, not verified:

- **ERC-20 `asset`** — a bounded 5-account CVL model (`erc20.spec`), reached through internal `SafeERC20Lib` summaries (`vaultv2_safe_erc20.spec`) that bypass the low-level `token.call`.
- **`IAdapter`** — `allocate` / `deallocate` are `NONDET` (adversarial adapter: returned ids / change are havoced); `realAssets()` is a per-adapter ghost bounded to `max_uint128` (`vaultv2_adapters.spec`).
- **`IAdapterRegistry`** — `isInRegistry` is a per-(registry, account) boolean ghost.
- **The four gates** — `canReceiveShares` / `canSendShares` / `canReceiveAssets` / `canSendAssets` are per-account verdict recorder ghosts (`vaultv2_gates.spec`).
- **`multicall`** — `NONDET DELETE` (the delegatecall paths are covered by direct parametric calls); `firstTotalAssets` (transient) is left unmirrored.

Types of Properties

Properties are categorized following the official Certora methodology.
Valid-State properties are **parametric** — they are automatically verified against every external function of the contract, including functions added after the specification is written.

Valid State — system-wide invariants that MUST always hold. They define the fundamental accounting and structural constraints of the vault. Once proven, they serve as trusted assumptions in other properties via `requireInvariant`, reducing verification complexity.

This engagement covers Valid State only. The remaining categories (Variable Transitions, State Transitions, High-Level, Reverts, Access Control, Reachability, Unit Tests) are out of scope for this run.

Verification Process

1. **Setup phase** — define the ghost mirrors, storage hooks, boundaries, and dependency models in CVL, and establish the harness and conf files. This phase addresses several prover limitations: the ERC-20 model, the adapter / gate / registry summaries, the internal `SafeERC20Lib` summary (the prover cannot inline its low-level `token.call`), and the transient `firstTotalAssets` (hooking transient storage is rejected by the prover, so it is left unmirrored).
2. **Crafting properties** — write the valid-state invariants over the `ghostTv*` mirrors and prove each by induction: a base case plus a preserved block per method that re-establishes the tight pre-state via `SETUP` / `requireInvariant`.

Assumptions

Assumptions are categorized into four groups: **Safe** (real-world constraints that do not reduce coverage), **Proved** (verified invariants reused as preconditions), **Unsafe** (scope reductions for tractability), and **Trusted** (initialization state assumed correct because constructor logic is outside the per-call model).

Safe Assumptions

Environment (`env.spec`):

- `e.msg.value == 0` (VaultV2 is not payable) and `e.msg.sender != 0`, `!= currentContract`.
- `e.block.timestamp ∈ [max_uint16, max_uint32)` — realistic timestamp window.
- `e.block.number != 0`.
- `requireSameEnv` pins the preserved-block env (block / timestamp / sender / value) to the rule env.

Domain (`vaultV2.spec`):

- `virtualShares ∈ [1, 1e18]` — the immutable `10**decimalOffset`, `decimalOffset ∈ [0, 18]`.
- `lastUpdate <= block.timestamp` — monotone time; the inductive step verifies, only the constructor base case is outside Certora's per-call time model.

ERC-20 (`erc20.spec`):

- Total supply equals the sum of the bounded account balances (no rebase / fee-on-transfer).
- Token decimals ∈ [6, 18]; the called token is never `address(0)`.

Proved Assumptions

The 10 proven valid-state invariants are carried into each rule's preserved block via `requireInvariant` (`setupValidStateVaultV2`), so every method sees a tight pre-state. See Valid State for the full list.

Unsafe Assumptions

- **Bounded share-holder set** (`vaultV2.spec`) — the vault share ledger is mirrored into the shared ERC20 ghosts keyed by `_VaultV2`, so vault-share holders reuse that token model's 5-distinct-account bound (`ghostErc20AccountsValues[_VaultV2][0..4]`) and the `sharesSolvency` sum (`SHARE_SUM()`) ranges over a fixed set.
- **Bounded ERC-20 account set** (`erc20.spec`) — balance / allowance

lookups are restricted to 5 distinct accounts per token.

- **Loop unrolling** capped at 2 iterations (`loop_iter: 3`), with `optimistic_loop` / `optimistic_hashing` enabled; `realAssets()` bounded to `max_uint128` so the `accrueInterestView` accumulation stays in range.

Trusted Assumptions

The constructor's initialization (`owner`, `decimals`, `virtualShares`, initial `lastUpdate`) is assumed complete: the immutables' bounds are taken as SAFE premises rather than re-verified against the constructor body, which is not modeled in CVL.

Verification Properties

System-wide invariants that hold at every reachable state of `VaultV2`, expressed over the `ghostTv*` storage mirrors and defined in `vaultV2_valid_state.spec` (the importable LIBRARY); `vaultV2_valid_state_run.spec` is the RUNNER verified by `valid_state.conf`.

Valid State

Property	Name	Description	Formula	Status
VS-VA-01	`maxRateBounded`	the curator-set interest rate cap stays within the protocol ceiling	`maxRate <= MAX_MAX_RATE`	✅
VS-VA-02	`relativeCapBounded`	every allocation id's relative cap stays within one WAD	`∀ id. relativeCap[id] <= WAD`	✅
VS-VA-03	`penaltyBounded`	every adapter's force-deallocate penalty stays within the ceiling	`∀ a. forceDeallocatePenalty[a] <= MAX_FORCE_DEALLOCATE_PENALTY`	✅
VS-VA-04	`performanceFeeBounded`	the performance fee stays within the protocol ceiling	`performanceFee <= MAX_PERFORMANCE_FEE`	✅
VS-VA-05	`managementFeeBounded`	the management fee stays within the protocol ceiling	`managementFee <= MAX_MANAGEMENT_FEE`	✅
VS-VA-06	`performanceFeeRecipientConsistency`	a non-zero performance fee always has a recipient set	`performanceFee != 0 => performanceFeeRecipient != 0`	✅
VS-VA-07	`managementFeeRecipientConsistency`	a non-zero management fee always has a recipient set	`managementFee != 0 => managementFeeRecipient != 0`	✅
VS-VA-08	`sharesSolvency`	the sum of vault-share balances never exceeds the tracked total supply — no over-issuance (CONSERVATION)	`Σ balanceOf (bounded 5-holder set) <= totalSupply`	✅
VS-VA-09	`sharesZeroAddressEmpty`	vault shares are never held by the zero address (SAFETY)	`balanceOf[0] == 0`	✅
VS-VA-10	`zeroCannotApprove`	the zero address never grants a share allowance (SAFETY)	`∀ s. allowance[0][s] == 0`	✅

Verification Results

All properties were executed per-rule on the local Certora Prover (`emv.jar`, version 8.16.2, solc 0.8.28) against the audit commit (`morpho-org/vault-v2@549b113`).

Category	Result
Valid State	10 ✅

A reachability smoke test (`debug/sanity_valid_state.conf`) witnesses that all 40 parametric methods are reachable from a valid state (no vacuous run).

Reuse / Import

The valid-state bundle is **importable**: a downstream verification project can assume VaultV2's valid state as trusted preconditions and reason over the exposed `ghostTv*` mirrors.

```
// consumer.spec
import ".../certora/specs/vaultV2_valid_state.spec";   // the LIBRARY, never the RUNNER

rule myProperty(env e, ...) {
    setupValidStateVaultV2(e);            // one-shot: assume VaultV2 is in a valid state
    // … or cherry-pick: requireInvariant maxRateBounded(e); requireInvariant sharesSolvency(e);
    require ghostTvPerformanceFee <= MAX_PERFORMANCE_FEE_CVL();
    ...
}
```

In the consumer conf, add `harnesses/VaultV2Harness.sol` to `files` and `link` it into the integrating contract. Import the **LIBRARY** (`vaultV2_valid_state.spec`), never the RUNNER (`vaultV2_valid_state_run.spec`, which would re-run the invariants). The full contract — setup functions, exposed mirrors, and inherited assumptions — is in `fv_docs/import_manifest.md`.

Setup and Execution

The bundle runs on the **local** Certora Prover (`certoraRun.py`, `emv.jar`) — no cloud key required. To use the cloud instead, add `--server production` and set `CERTORAKEY`.

```
# Prove the valid-state invariants (base case + every method)
certoraRun.py certora/confs/valid_state.conf

# Reachability smoke test — all 40 parametric methods reachable from a valid state
certoraRun.py certora/confs/debug/sanity_valid_state.conf

# Typecheck / compile only
certoraRun.py certora/confs/valid_state.conf --compilation_steps_only
```

Bundle layout:

```
certora/
├── harnesses/VaultV2Harness.sol               # VaultV2Harness is VaultV2 (forwards ctor)
├── specs/
```

```
│   ├── setup/
│   │   ├── env.spec                          # setupEnv, requireSameEnv, timestamp bounds
│   │   ├── erc20.spec                        # bounded 5-account ERC20 model (asset + vault shares)
│   │   ├── vaultV2.spec                      # methods{}, ghost mirrors + hooks, setupVaultV2
│   │   ├── vaultv2_safe_erc20.spec           # internal SafeERC20Lib summaries
│   │   └── vaultv2_gates.spec / vaultv2_adapters.spec  # gate / IAdapter / IAdapterRegistry summaries
│   ├── debug/vaultV2_sanity_valid_state.spec   # sanityValidState (reachability)
│   ├── vaultV2_valid_state.spec                # LIBRARY — invariants + setup aggregators (import
this)
│   └── vaultV2_valid_state_run.spec            # RUNNER  — use invariant … (verified by
valid_state.conf)
└── confs/
    ├── valid_state.conf                      # verifies the 10 invariants
    └── debug/sanity_valid_state.conf         # reachability smoke test
```

Resources

- Certora Tutorials — Official Certora documentation and guided tutorials
- AlexZoid FV Resources — Curated collection of formal verification resources, examples, and references
- Updraft Assembly & Formal Verification Course — Comprehensive video course covering assembly and formal verification from the ground up
- RareSkills Certora Book — Structured tutorial covering CVL syntax, patterns, and common pitfalls